COSC 39: Theory of Computation

Credit Statement: Talked to Carson Bates'27 and Kiran Jones'27.

Problem 1. Write down the decision version of the following problems, then describe a polynomialtime self-reduction assuming an oracle to the decision problem.

FINDMAXCLIQUE

- **Input:** An undirected graph G.
- **Output:** Compute a maximum-size clique in G.

Solution.

We will describe an oracle reduction to the following decision problem.

MAXCLIQUE

- **Input:** An undirected graph $G = (V, E)$ and an integer k.
- **Output:** Does G contain a clique of size at least k?

First, we determine the maximum clique size. Starting from $k = |V|$ and decrementing k by one each time, we query the oracle on (G, k) until the oracle answers YES. Let $k^\star$ be the first value for which the oracle answers YES. Then G contains a clique of size $k^\star$, and no clique larger than $k^\star$ exists.

Next, we recover an actual clique of size $k^\star$. We define a new graph H and initialize it to G, i.e., $H := G$. While $|V(H)| > k^\star$, do the following:

1. Pick any vertex $v \in V(H)$.
2. Query the oracle on $(H - v, k^\star)$, where $H - v$ denotes the graph obtained by deleting v.
   (a) If the oracle answers YES, then there exists a clique of size $k^\star$ not using v, so we permanently delete v and set $H := H - v$.
   (b) If the oracle answers NO, then every clique of size $k^\star$ must contain v, so we keep v.

At termination, the remaining graph has exactly $k^\star$ vertices. Since the oracle invariant guarantees that it still contains a clique of size $k^\star$, those remaining vertices must form a clique. Hence we have recovered a maximum clique.

We make at most $|V|$ oracle queries to determine $k^\star$, and at most $|V|$ additional oracle queries during reconstruction. Since all other computations are polynomial-time graph operations, this is a polynomial-time self-reduction.

Problem 2.
Write down the decision version of the following problems, then describe a polynomialtime self-reduction assuming an oracle to the decision problem.

FIND3COLORING

- **Input:** An undirected graph G.
- **Output:** Compute a proper vertex 3-coloring of G (if exists).

Solution.

We will describe an oracle reduction to the following decision problem.

3COLORING

- **Input:** An undirected graph $G = (V, E)$.
- **Output:** Does G have a 3 coloring?

We use the following algorithm to first determine if G has a 3 coloring, and find it if it does.

1. If the oracle answers NO, then G is not 3-colorable, so we terminate.
2. If the oracle answers YES, then we recover an explicit coloring as follows.

Introduce three new vertices r, b, g and connect them to form a triangle. In any proper 3-coloring, these three vertices must receive distinct colors, which we label red, blue, and green respectively.

Now, for each vertex $v \in V(G)$, we determine its color one at a time.

To determine the color of any vertex v, we :

1. Construct a graph G_r by adding an edge between v and both b and g. Then v can only be colored red.

   Query the oracle on G_r.

   (a) If the oracle answers YES, then there exists a valid 3-coloring with v colored red, so we permanently assign v the color red.

   (b) If the oracle answers NO, then v cannot be colored red.

2. If red failed, repeat the same procedure forcing v to be blue by connecting v to r and g.

(a) If the oracle answers YES, assign v the color blue.

(b) Otherwise, assign v the color green.

In every future oracle query, we enforce these choices by adding edges to the other two colors so that each previously colored vertex is forced to keep its assigned color. However, note that we do not query the oracle with a parital coloring of the graph.

Since each vertex requires at most two oracle queries, we make at most $2|V| + 1$ oracle queries and since all graph modifications can be performed in polynomial time, this is a polynomial-time self-reduction.